# Redux Toolkit – Complete Beginner Documentation

## 1. Introduction

When React applications start growing, managing data becomes messy.

At first things look simple.

A component holds some state.

Then that state needs to be shared with another component.

So we pass it as props.

This is called **props drilling**.

Example flow:

`App → Parent → Child → GrandChild`

Even if only the **GrandChild** needs the data, we still pass it through every layer.

As applications grow, this becomes painful.

Problems start appearing:

- Too many props
  - Difficult debugging
  - State scattered everywhere
  - Hard to scale applications

To solve this, developers started using **centralized state management**.

This is where **Redux** came in.

---

# 2. History of Redux

Redux was created in **2015** by:

- **Dan Abramov**
  - **Andrew Clark**

Redux was inspired by the **Flux architecture** created at Facebook.

The main idea was simple:

Instead of storing state inside many components, keep **one central store** for the entire application.

So the architecture became:

`Components → Dispatch Action → Store → Update State → UI Updates`

Redux introduced three core principles:

## 1. Single Source of Truth

The entire application's state lives inside **one store**.

Example:

```
store = {
  user: {},
  cart: [],
  theme: "dark"
}
```

Everything is stored in one place.

---

## 2. State is Read Only

You cannot modify the state directly.

Instead you send **actions**.

Example:

```
dispatch({
  type: "ADD_TODO",
  payload: "Learn Redux"
})
```

---

## 3. Changes are made with Pure Functions

Redux uses **reducers** to update state.

Reducers are just functions.

Example:

```
function reducer(state, action) {
  if(action.type === "ADD_TODO") {
    return {...state, todos: [...state.todos, action.payload]}
```

```
  }
}
```

Reducers take:

```
current state
+
action
=
new state
```

---

# 3. Problems with Classic Redux

Redux was powerful, but it had a **big problem**.

It required **too much boilerplate code**.

To create even a small feature you had to write:

1. Action Types
2. Action Creators
3. Reducers
4. Store configuration
5. Middleware setup

Example structure:

```
actions.js
actionTypes.js
reducers.js
store.js
constants.js
```

A small feature could require **100+ lines of code**.

Developers started complaining.

Redux became famous for being:

"Powerful but painful."

This is where **Redux Toolkit** enters the story.

---

# 4. What is Redux Toolkit

Redux Toolkit (RTK) is the **official recommended way to write Redux logic today**.

It was introduced by the Redux team in **2019**.

Redux Toolkit simplifies Redux development and removes most of the boilerplate code.

The goal of Redux Toolkit is simple:

**Make Redux easier, safer, and faster to write.**

Instead of writing many files manually, Redux Toolkit gives **utility functions** that automatically generate the required logic.

---

# 5. Why Redux Toolkit was Created

Redux Toolkit was created to solve these problems:

## Problem 1

Too much boilerplate.

Solution:

RTK automatically generates **actions and reducers**.

---

## Problem 2

Complex store configuration.

Solution:

RTK provides **configureStore()**.

---

## Problem 3

Accidental state mutations.

Solution:

RTK uses **Immer.js** internally to allow safe state mutation syntax.

---

## Problem 4

Async logic was confusing.

Solution:

RTK provides **createAsyncThunk()**.

---

### Problem 5

Too many libraries required.

Solution:

RTK bundles the most important Redux tools together.

---

# 6. Core Idea of Redux Toolkit

Redux Toolkit follows the same Redux architecture.

```
Component
   ↓
Dispatch Action
   ↓
Reducer updates state
   ↓
Store updates
   ↓
UI re-renders
```

But the difference is:

**Redux Toolkit reduces the amount of code needed to implement this architecture.**

---

# 7. Installation

Install Redux Toolkit and React Redux.

```
npm install @reduxjs/toolkit react-redux
```

Redux Toolkit already includes Redux internally.

So you **do not need to install redux separately.**

---

# 8. Important Concepts in Redux Toolkit

There are a few important building blocks.

### 1. Store

The central place where all application state lives.

### 2. Slice

A slice represents a **portion of the state**.

Example:

```
userSlice
cartSlice
themeSlice
```

Each slice contains:

- state
  - reducers
  - actions

---

### 3. Reducer

A function that updates the state.

---

### 4. Action

An object describing what happened.

Example:

```
{
 type: "cart/addItem",
 payload: item
}
```

---

### 5. Dispatch

A function used to send actions to the store.

---

### 6. Selector

A function used to read data from the store.

---

# 9. configureStore()

In classic Redux, creating a store required multiple steps.

Redux Toolkit simplifies it using **configureStore()**.

Purpose:

- Create store
  - Combine reducers
  - Enable Redux DevTools
  - Add middleware automatically

Example:

```
import { configureStore } from "@reduxjs/toolkit"
import counterReducer from "./counterSlice"

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
})
```

Now the store is ready.

---

# 10. createSlice()

This is the **most important feature of Redux Toolkit**.

A slice combines:

- state
  - reducers
  - actions

into a single place.

Example:

```
import { createSlice } from "@reduxjs/toolkit"

const counterSlice = createSlice({
  name: "counter",

  initialState: {
    value: 0
  },

  reducers: {
```

```
    increment: (state) => {
      state.value += 1
    },

    decrement: (state) => {
      state.value -= 1
    }
  }
})

export const { increment, decrement } = counterSlice.actions
export default counterSlice.reducer
```

What Redux Toolkit automatically generates:

- Action Types
  - Action Creators
  - Reducers

All automatically.

---

# 11. How State Mutation Works (Immer)

In classic Redux, state must **never be mutated**.

Example:

❌ Wrong

```
state.count += 1
```

✅ Correct

```
return {
  ...state,
  count: state.count + 1
}
```

But Redux Toolkit allows mutation-like syntax.

Example:

```
state.count += 1
```

This works because Redux Toolkit internally uses **Immer.js**.

Immer converts mutation syntax into immutable updates behind the scenes.

So the state is still safe.

---

# 12. Connecting Redux Toolkit with React

Redux works with React through **React Redux**.

React Redux provides:

```
Provider
useDispatch
useSelector
```

---

## Provider

Wrap the application.

```
<Provider store={store}>
   <App />
</Provider>
```

---

## useDispatch()

Used to send actions.

Example:

```
const dispatch = useDispatch()
dispatch(increment())
```

---

## useSelector()

Used to read state from store.

Example:

```
const count = useSelector((state) => state.counter.value)
```

---

# 13. createAsyncThunk()

Redux Toolkit also handles **API requests and async operations**.

For example:

- Fetch data from server
  - Login user
  - Load products

Redux Toolkit provides **createAsyncThunk()**.

It automatically generates:

```
pending
fulfilled
rejected
```

Example concept:

```
fetchData
   ↓
pending
   ↓
success → fulfilled
error → rejected
```

Reducers can handle these states.

---

# 14. Folder Structure (Recommended)

A clean Redux Toolkit project structure looks like this:

```
src
├── app
│     store.js
│
├── features
│     counter
│        counterSlice.js
│
├── components
│
└── App.jsx
```

Each feature keeps its own slice.

This is called **feature-based architecture**.

---

# 15. Advantages of Redux Toolkit

Redux Toolkit provides many benefits.

## 1. Less Boilerplate

You write far less code compared to classic Redux.

---

## 2. Built-in Best Practices

Redux Toolkit automatically includes:

- Redux DevTools
  - Middleware
  - Immutable state checks

---

### 3. Easier Async Logic

createAsyncThunk simplifies API calls.

---

### 4. Better Project Structure

Slices keep related logic together.

---

### 5. Safer State Updates

Immer prevents accidental mutation.

---

### 6. Official Redux Standard

Redux Toolkit is now the **recommended way to use Redux**.

---

# 16. Redux vs Context API

Many beginners ask this question.

When should we use Redux?

### Context API

Good for:

- small applications
  - theme management
  - authentication state
  - simple global data

---

### Redux Toolkit

Better for:

- large applications
  - complex state
  - multiple features sharing data
  - predictable state updates

---

# 17. Performance Benefits

Redux Toolkit improves performance by:

- preventing unnecessary re-renders
  - centralizing state updates
  - enabling selective subscriptions through selectors

React components only re-render when the selected state changes.

---

# 18. Redux DevTools

Redux DevTools allow developers to:

- see every action dispatched
  - inspect state changes
  - time travel debugging
  - track bugs easily

Redux Toolkit automatically enables DevTools in development.

---

# 19. When Should You Use Redux Toolkit

Redux Toolkit is useful when:

- many components share the same data
  - state logic becomes complex
  - application grows large
  - debugging state becomes difficult

---

# 20. Summary

Redux Toolkit is a modern state management library built on top of Redux.

It simplifies Redux development by reducing boilerplate and providing powerful utilities.

Key points:

- Redux was created in 2015
  - Redux Toolkit was introduced in 2019
  - Redux Toolkit is now the **official way to write Redux**

Main tools provided by Redux Toolkit:

- configureStore()
  - createSlice()
  - createAsyncThunk()
  - built-in middleware
  - Immer integration

Redux Toolkit helps developers write scalable, maintainable, and predictable state management logic.